

IMPERIAL

Instant Policy ++

In-Context Imitation Learning

Teoh Kai Shun

16/09/2025

By the end of the Presentation :

- What is In-Context Imitation Learning and Why is it important?
- What is Hyperbolic Space and why is it important?
- What is Instant Policy and How Non-euclidean methods can be utilised to improve Instant Policy?

In-Context Imitation Learning

In Context Imitation Learning

What is it?

- A learning paradigm where a model adapts to a new task directly from demonstrations, *without retraining*
- **In-Context Imitation Learning (ICIL)** is a combination of 2 forms of Learning :
 - **In-Context Learning (ICL)**
 - **Imitation Learning (IL)**
- Putting it all Together:
 - **ICL** => Learns **Context** => Model can identify **Context** from demonstrations and use it for predictions
 - **IL** => Learns **Actions** => Model knows how to perform 1 task
 - **(ICL + IL) ICIL** => Learns **Context** and *uses* it to predict **Actions** => Model learns to identify **Context** (from demos) and learns to ***perform tasks under this Context*** => Can perform any tasks by just giving model demonstrations

In Context Imitation Learning

Why ICIL? (1)

- Within Robotics **ICIL** is seen as an upgrade to **IL**
 - While **IL** has been successful within robotics [1] , it faces these limitations [2]:
 - Require a lot of robot demonstrations (Expensive, limited in quantity, proprietary)
 - Single-tasks policy (needs retraining, can't reuse data)
 - **ICIL** aims to bridge this gap
 - ‘**Learn** from demos’ => Less data
 - This also implies that policy is now **context-based**.
 - Can solve tasks within the context it is trained on.
 - When solving new tasks, no longer need retraining, just demonstrations.

1. Argall, B. D., Chernova, S., Veloso, M., & Browning, B. (2009). *A survey of robot learning from demonstration*. Robotics and Autonomous Systems, 57(5), 469–483.
2. Hussein, A., Gaber, M. M., Elyan, E., & Jayne, C. (2017). *Imitation learning: A survey of learning methods*. ACM Computing Surveys, 50(2), 1–35.

In Context Imitation Learning

Why ICIL? (2)

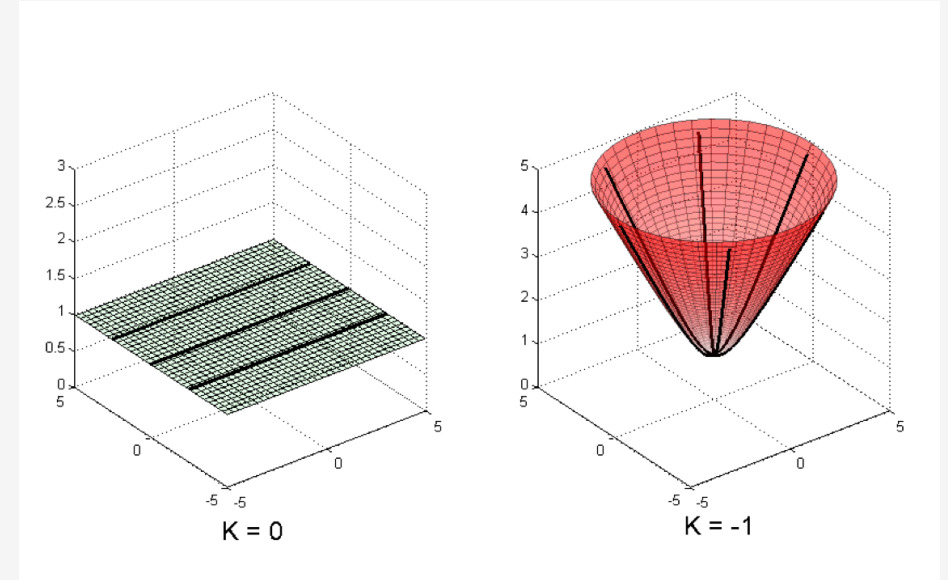
- **ICIL** can potentially change the use of robots
 - Quick adaptation just from demos makes it a flexible worker
 - Can take on more responsibility
 - Eg: Packaging : “Shapes and size of packages changes, what to do when new shapes/sizes arises?”
 - Old (Use IL) : Collect training data on new packages, retrain model, deploy
 - Time-consuming, require many data on new package (might not be feasible).
 - New (Use ICIL): Give demos with new package, model learns from demo
 - Fast and intuitive* learning, very integrable into production lines

Hyperbolic Space

Hyperbolic Space

What is it?

- **Hyperbolic space** is a Manifold with constant negative curvature
 - Due to its negative curvature, it has the property of **exponential volume growth**.
 - This causes its geometry to take on a **continuous tree structure**
- **Euclidean space** on the other hand is a Manifold with **zero-curvature**
- Due to its zero-curvature, it has the property of **linear volume growth**
 - This causes its geometry to take on a **grid-like structure**



Hyperbolic Space

Why is Hyperbolic Space important

1. Nickel, M., & Kiela, D. (2017). Poincaré embeddings for learning hierarchical representations. Advances in Neural Information Processing Systems (NeurIPS), 30.
2. Chami, I., Ying, Z., Ré, C., & Leskovec, J. (2019). Hyperbolic graph convolutional neural networks. Advances in Neural Information Processing Systems (NeurIPS), 32.

- Recent research has shown Hyperbolic Space to be a promising embedding space
 - Capable of embedding data with tree-like structure at low distortion and dimensions
 - Memory efficient!
- Interest in Hyperbolic Embeddings spurred researchers towards Hyperbolic Neural Network components
 - These components are better able to leverage geometry induced-biased within hyperbolic embeddings by utilising geometry-preserving functions
- SOTA performance [1,2]

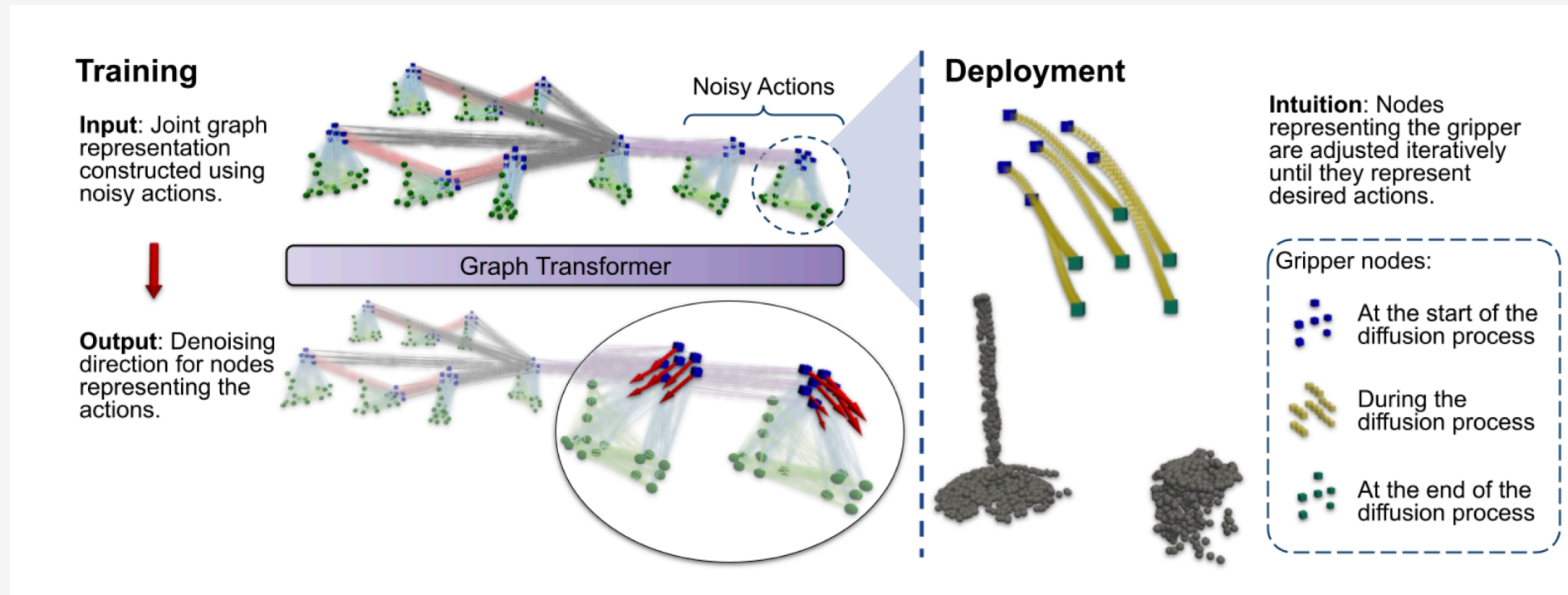
Instant Policy

Instant Policy

Instant Policy : Introduction

1. Vosylius V and Johns E. Instant Policy: In-Context Imitation Learning via Graph Diffusion. 2025. arXiv: 2411.12633 [cs.RO]. Available from: <https://arxiv.org/abs/2411.12633>

- Instant Policy : SOTA; frames ICIL as a graph generation problem [1]
- Intuitively, instead of predicting actions, the model learns how to re-adjust agent's position after a noisy action is taken
 - Through these adjustments, the clean actions are derived from the noisy actions.



Instant Policy

Instant Policy : Graph Representation

- To frame ICIL as a Graph generation problem, Instant Policy defines graph representations for Demos, Observations and Actions

- Graph representations for Observations are called **Local Graph**

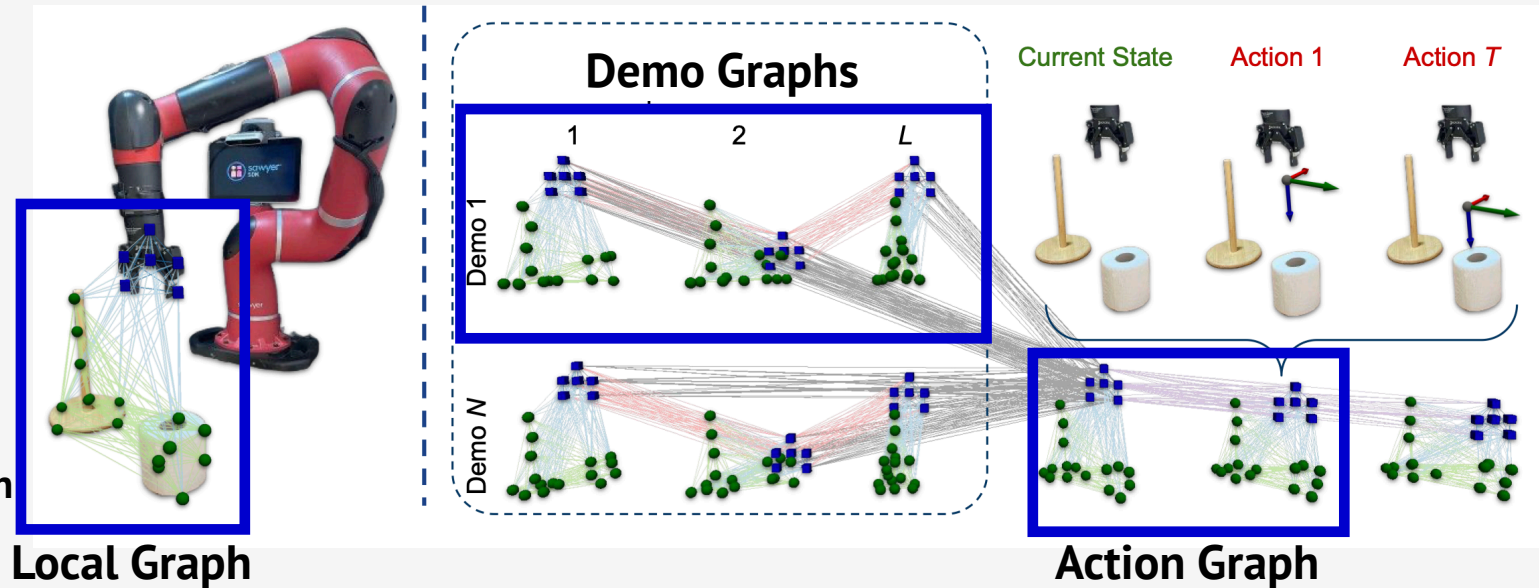
- Agent Nodes + Scene Nodes
- Edges between Agent and Scene Nodes

- Graph representations for Demos are called **Demo Graph**.

- Agent Nodes across Demos
- Edges between corr. Agent Nodes

- Graph Representations for Actions are called **Action Graphs**

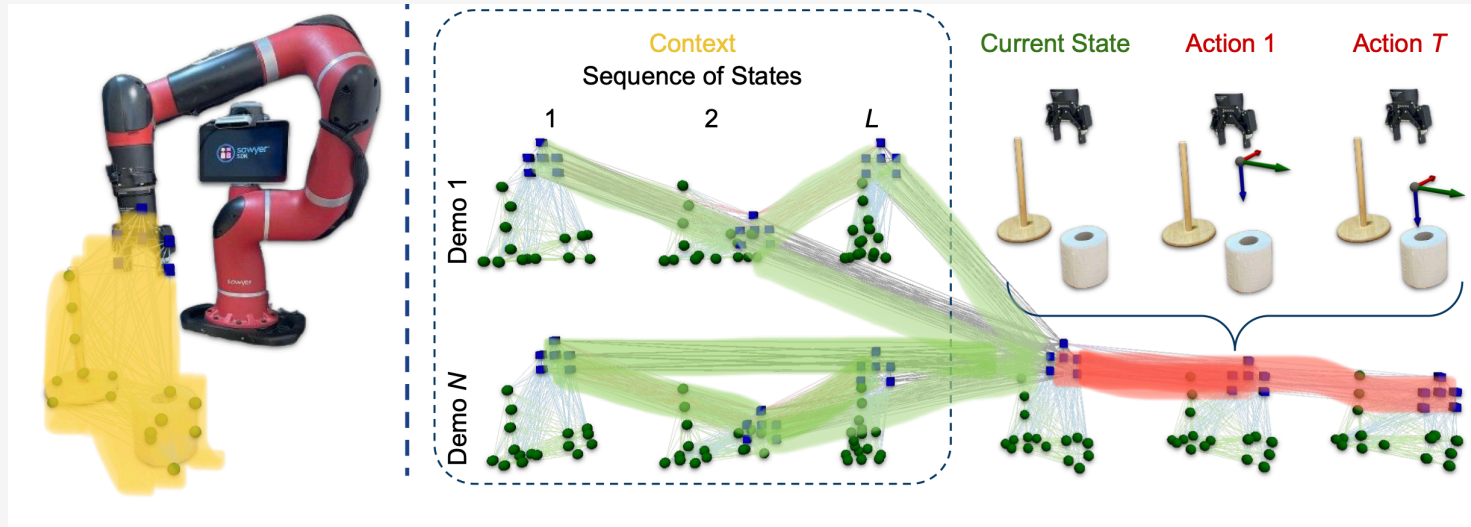
- Consist of Agent nodes of 2 **Local Graphs**
 - Current Obs
 - Obs post-action
- Edge between corr. Agent Nodes



Instant Policy

Instant Policy : Graph Diffusion

- Instant Policy makes adjustment predictions via graph diffusion.
- Instant Policy graph diffusion process can be broken down into 3 component:
 - Rho Network : Responsible for aggregating scene information to agent nodes' features (Acts on **LG**)
 - Phi Network : Responsible for aligning current observation to demonstrations (Acts on **DG + Current LG**)
 - Psi Network : Responsible for aligning current observation with future states (Acts on **AG**)



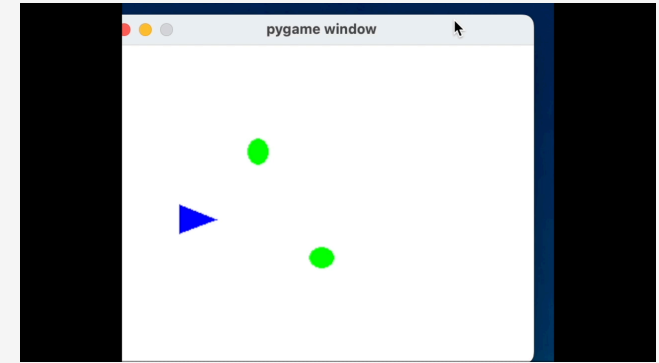
Colours show what network acts on which subgraph

- Yellow (Rho)
- Green (Phi)
- Red (Psi)

Instant Policy

Instant Policy : Limitations

- Manual Demonstration processing required to make model perform well
 - Not a robust solution (Hard to define a procedure to process all Demos)
 - If not done well, prone to oscillating behaviour
- Single embedding space for both local spatial information (within each observation) and global temporal relation (within each actions)
 - Although they are related, they are fundamentally capturing different semantics
 - Spatial : ***Where is the agent right now?***
 - Temporal : ***How does the agent move across time?***
 - Forcing 2 semantics of different scope into a single embedding requires high embedding dimensions such that the embedding is sufficiently expressive.
 - Memory + Computationally Expensive on model



Proposed Improvements

Proposed Improvements

- If **actions** and **observations** both encapsulate different semantic information, why not process them differently?
 - Thus, my model propose to utilise 2 channels that embeds each input individually:
 - Hyperbolic Channel : Actions
 - Euclidean Channel : Observations
- These 2 channels of information are then combined within a **Product Manifold** which **preserves the complementary inductive biases** of Euclidean and Hyperbolic spaces [1]
 - This enables effective alignment between current observations and demonstrations.

1. Gu, A., Sala, F., Gunel, B., & Ré, C. (2019). Learning mixed-curvature representations in product spaces. International Conference on Learning Representations (ICLR).

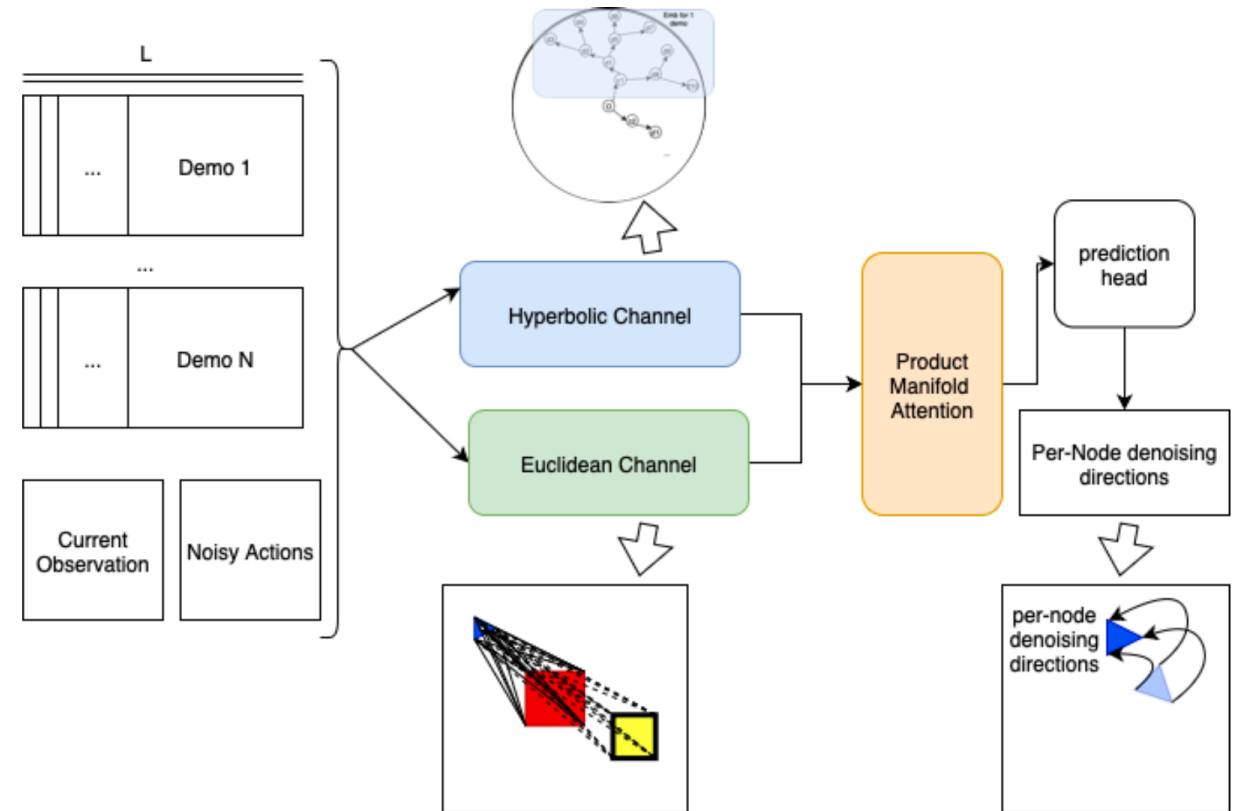


Figure 1.1: High-level overview of Agent

Proposed Improvements : Why 2 channels?

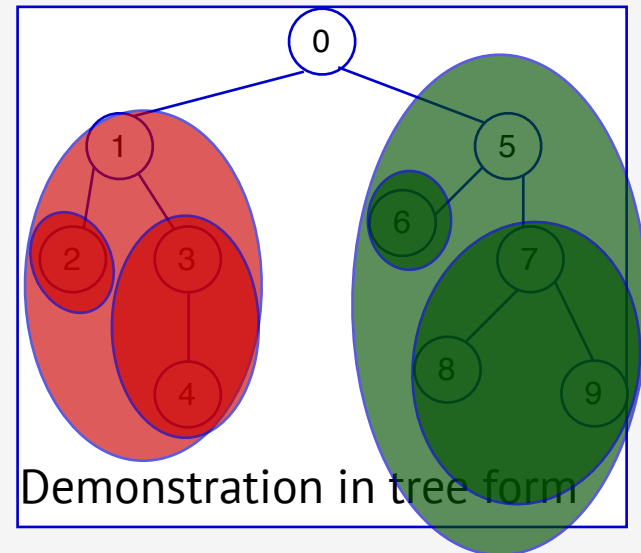
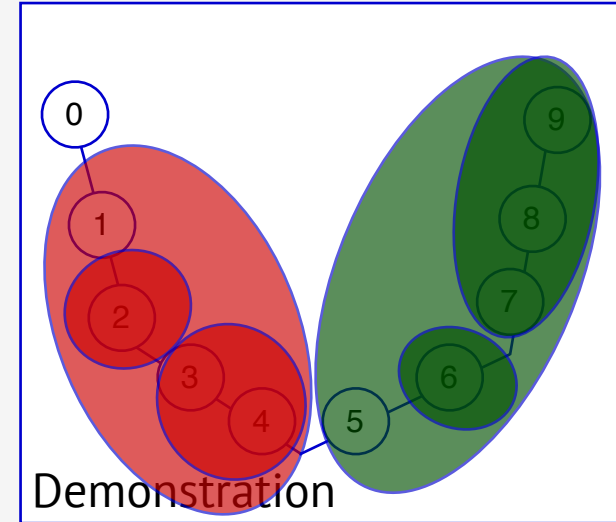
Limitations of 1 Channel embedding

- Previously, we mention that by forcing 2 channels of information (Actions and Observations) within 1 single embeddings space, embedding space will generally **require higher dimension** to capture complex relations between these channels.
 - Might be **more memory efficient to individually encode** each channel of information, then combine them at a later stage
- By limiting ourselves to 1 embedding space, we are being **ignorant to the suitability of embedding space** for each channels
 - Ideally, embedding space should be chosen such that its **geometry reflect the underlying relations** within an individual source of information
 - This results in **more efficient and accurate representation**
 - Furthermore, choice of embedding space serve as an **option to encode priors** within model
 - Eg if we know actions are hierarchical, we explicitly use hyperbolic embeddings for actions

Proposed Improvements : Hyperbolic Space (1)

Actions as semantics tree

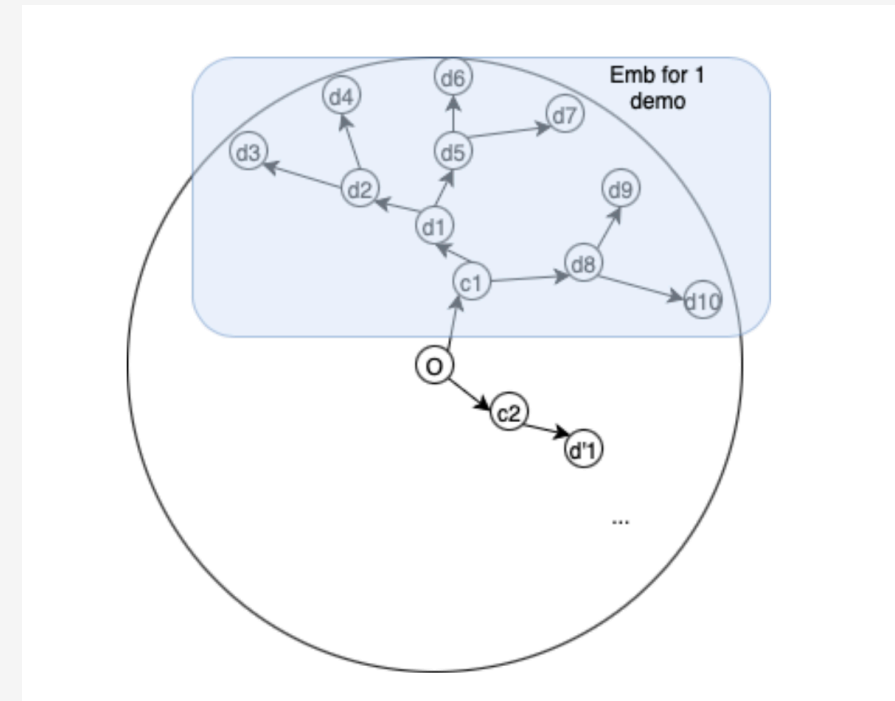
- Demonstrations can be broken into **waypoints = semantic states**
- Waypoints can be refined into finer sub-steps → hierarchy
 - Local neighbourhood of each waypoint defines this set of finer waypoints
- In 2D tasks, transitions = changes in orientation or agent state
- Recursive clustering of these transitions → **tree structure**
- Tree encodes both transitions (cluster ↔ cluster) and 'specificity of instructions' (depth)



Proposed Improvements : Hyperbolic Space (2)

Linking Current Observations/Post-action Observations to Demonstrations

- Now that we have mention how actions within demos can be represented as a Tree, how do we represent Current/Post-action observations within this Tree?
 - Linking them is extremely important, as they serve as the 'point of reference' for trees constructed from each demo
 - Without this link, trees will not be relevant to current/post-action observations, rendering them useless
- Solution: insert current observation before each demo
- Motivation from *Coarse-to-Fine IL*: replaying from current state = replaying demo [1]
- Anchoring makes trees relevant to current/post-action observations

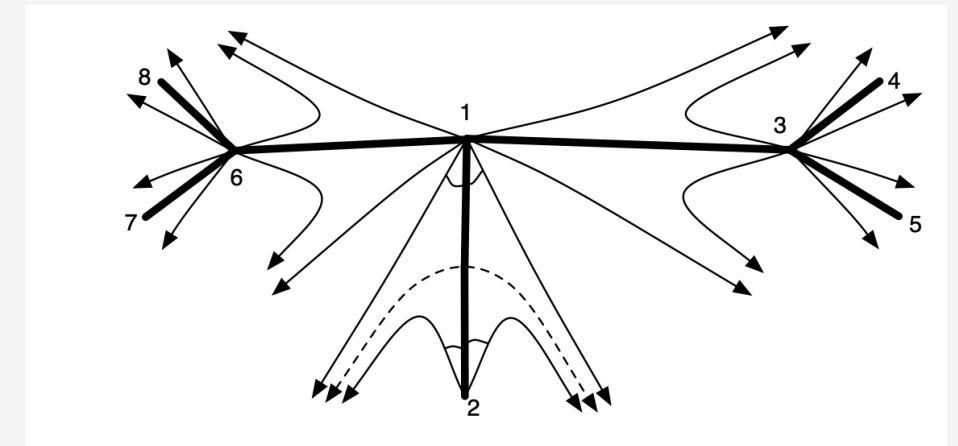


Proposed Improvements : Hyperbolic Space (3)

Hyperbolic Embedding for Actions : Why and How?

- Euclidean embeddings need **high dimensions** to capture hierarchies → inefficient
- Hyperbolic embeddings = **exponential growth** → compact & memory efficient [1]
 - Proven: **2D hyperbolic** can embed trees with low distortion [2]
- We use **Sarkar's Constructor** [3] to embed action trees
 - Training-free, systematic procedure
 - Learning Hyp Emb can be computationally expensive
 - Embeddings explicitly conditioned on **demos + current obs**

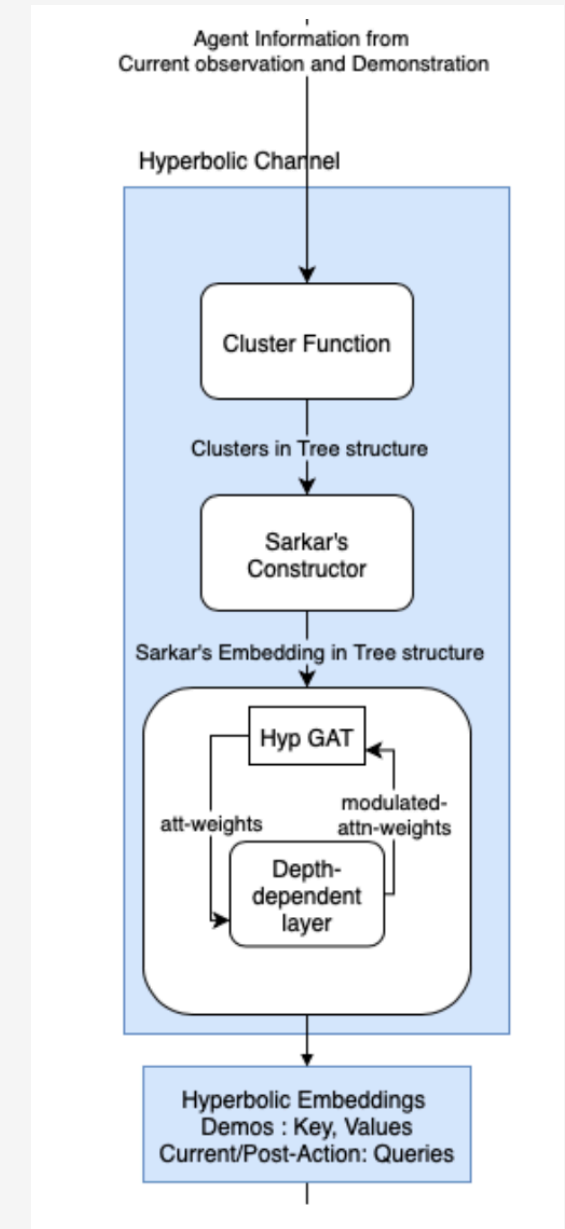
1. Sala, F., De Sa, C., Gu, A., & Ré, C. (2018). Representation tradeoffs for hyperbolic embeddings. International Conference on Machine Learning (ICML).
2. Bansal S and Benton A. Comparing Euclidean and Hyperbolic Embeddings on the WordNet Nouns Hypernymy Graph. 2021.
3. Sarkar, R. (2011). Low distortion Delaunay embedding of trees in hyperbolic plane. Graph Drawing (GD), Lecture Notes in Computer Science, 7034, 355–366.



Proposed Improvements : Hyperbolic Space (4)

How Hyperbolic Space? (2) : HGAT for temporal awareness

- Hyperbolic embeddings already **encode temporal order** via tree structure
- We add Hyperbolic Graph Attention (HGAT) for robustness
 - Depth-dependent linear gate → induces hierarchical bias
 - Enables **mixing across levels** (inter-/intra-level)
- Makes the hyperbolic channel **robust to mis-clustering**
- Preserves hierarchical + temporal awareness



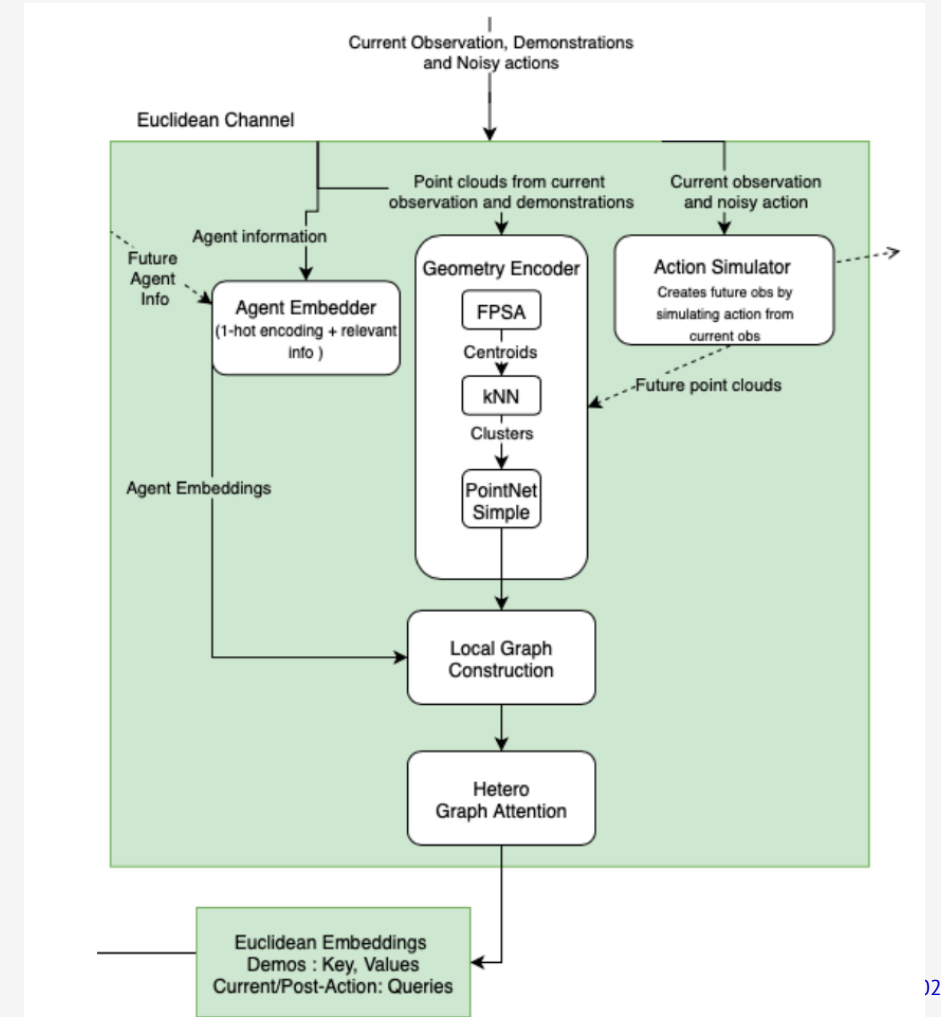
Euclidean Embeddings: Adapting from Instant Policy

For Completeness

- Our Euclidean channel is an adaptation of Rho in Instant Policy



- Encodes local geometry of scene
- Anchors actions to physical state
- Temporally agnostic (just spatial grounding)

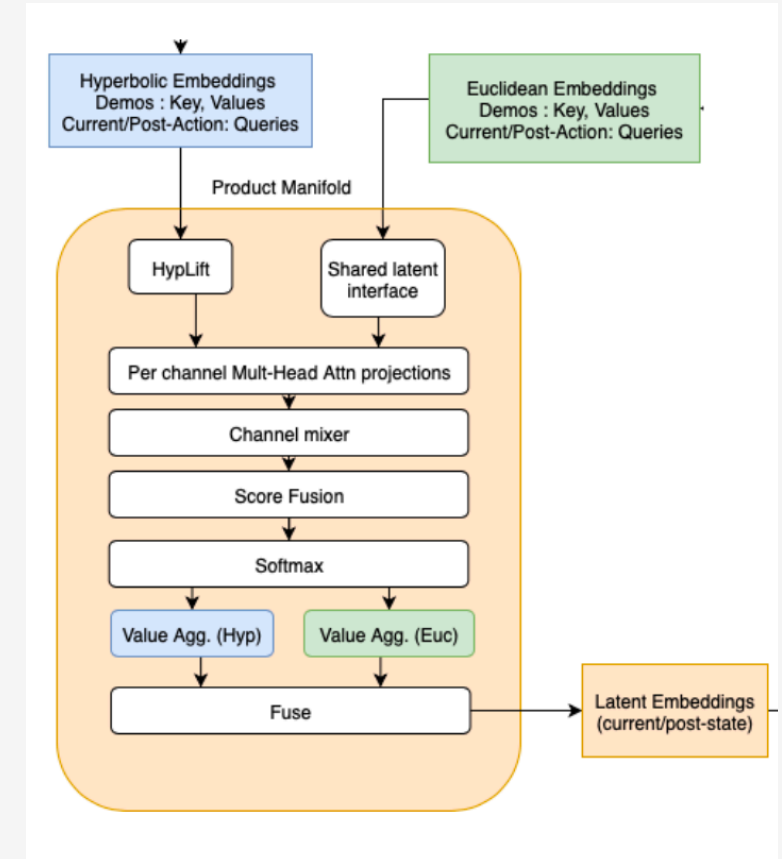


Proposed Improvements : Product Manifold

The Bridge

- Actions and observations encode **different semantics**
- Forcing both into **one embedding space** = high dimension, inefficient
- Instead: embed separately => Euclidean (scene), Hyperbolic (actions)
- Combine in a Product Manifold
- Preserves complementary inductive biases of each space
- Enables effective alignment between current obs and demos

Just Concat!



$$p = (p_1, p_2 \dots p_k), \text{ where } p_i \in \mathcal{M}_\gamma$$

$$v = (v_1, v_2 \dots v_k), \text{ where } v_i \in T_{p_i} \mathcal{M}_\gamma$$

Proposed Improvements : Product Manifold

Context Alignment (Attention)

Given a current state $i=\text{cur}$ and a set $\mathcal{D}$ of demo items (frames or nodes), we form *per-channel* queries, keys and values:

$$Q^E = A_Q^E \hat{\phi}_{\text{cur}}^E, K^E = A_K^E \hat{\phi}_j^E, V^E = A_V^E \hat{\phi}_j^E \quad \text{and} \quad Q^H = A_Q^H \hat{\phi}_{\text{cur}}^H, K^H = A_K^H \hat{\phi}_j^H, V^H = A_V^H \hat{\phi}_j^H,$$

- Attention aligns current obs & demo trajectories
 - Euclidean channel = precise spatial localisation (Green)
 - Hyperbolic channel = hierarchical action semantics (Blue)

$$s_{ij}^E = \frac{\langle Q_i^E, K_j^E \rangle}{\sqrt{d_h}}, \quad s_{ij}^H = \frac{\langle Q_i^H, K_j^H \rangle}{\sqrt{d_h}},$$
$$\lambda^E, \lambda^H = \text{softmax}([\gamma^E, \gamma^H]) \Rightarrow s_{ij}^P = \lambda^E s_{ij}^E + \lambda^H s_{ij}^H.$$

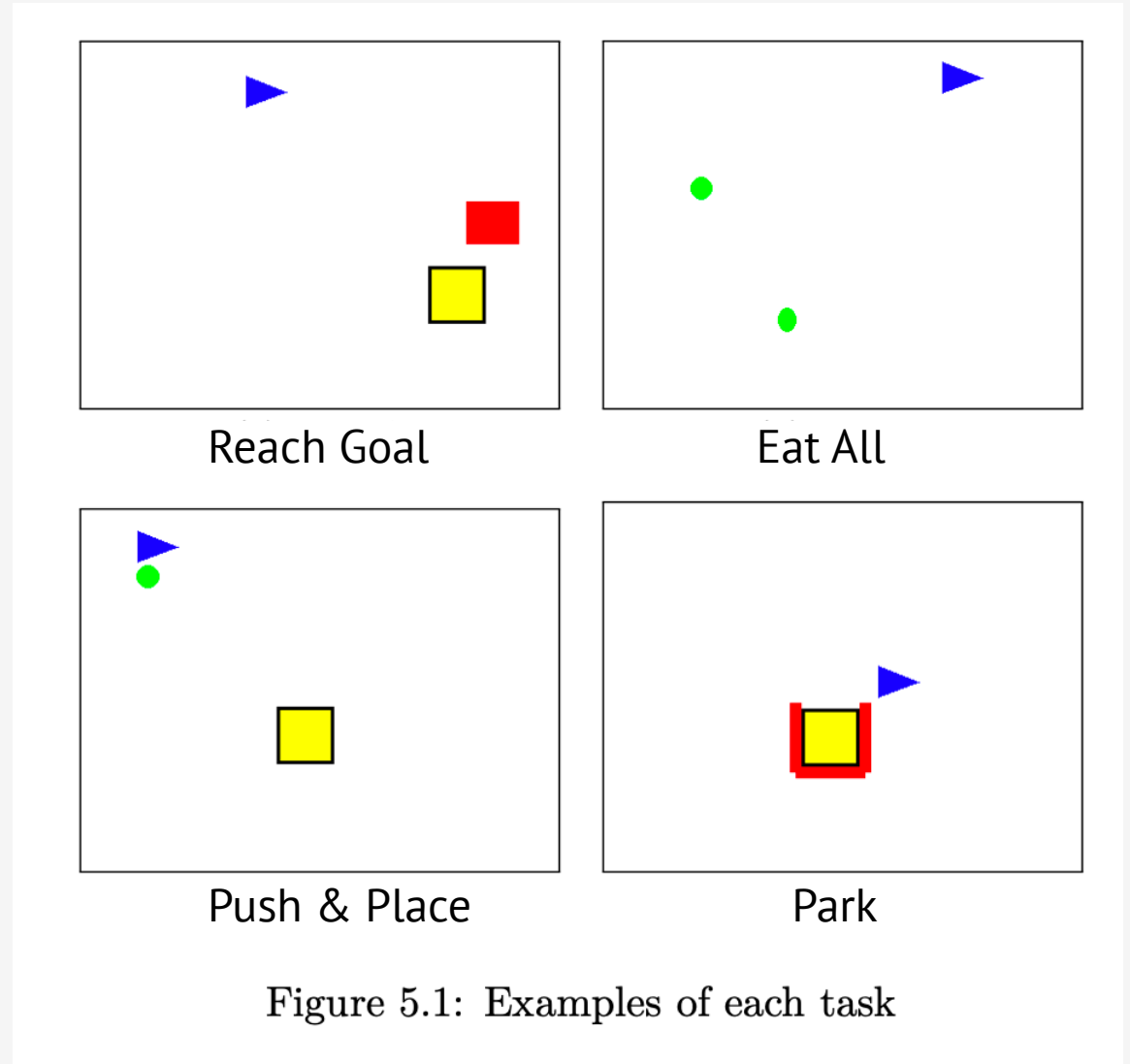
- Joint attention enables context-aware action selection
- Improves ICIL robustness compared to single-channel embedding

Experiments

Experiment 1: Baseline Performance

Experimental Setup

- Environment: 2D pseudo-game tasks (Reach Goal, Eat All, Push & Place, Parking)
- Trained with pseudo-demonstrations only
- Baseline Model: Instant Policy
- Purpose: Test if separating channels improves ICIL performance



Experiment 1: Baseline Performance

Results

- Reach Goal: 2/10 (ours) vs 5/10 (IP)
 - Eat All: 1/10 (ours) vs 3/10 (IP)
 - Push & Place: 0/10 both
 - Parking: 0/10 both
-
- **Observation:** dual-channel did not surpass IP on tasks
-
- **Why it matters:** Instant Policy still stronger at **step-level correction**
 - Our separation didn't give task-level gain

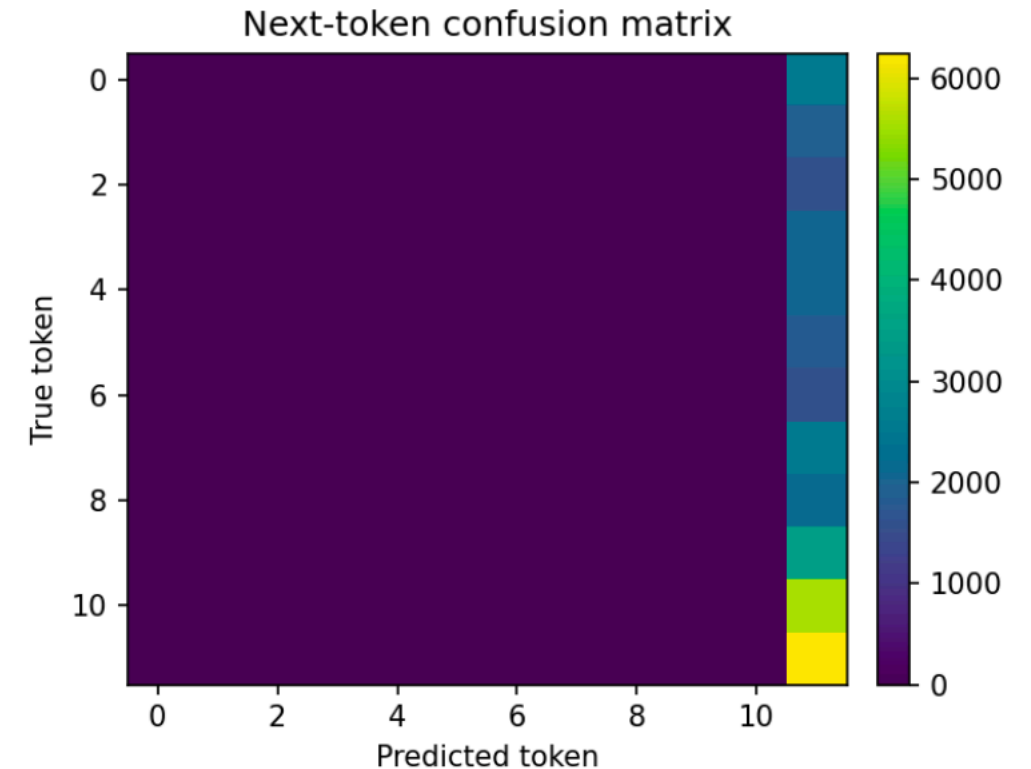
Task	Proposed Model	Instant Policy
A: Reach Goal	2/10	5/10
B: Eat All Edibles	1/10	3/10
C: Push & Place	0/10	0/10
D: Parking	0/10	0/10

Experiment 2 : Did Action Semantic work?

Action Semantics Probe

- What we did & why
 - Hypothesis: Hyperbolic embeddings capture **next-action semantics**
 - Tested with linear probe (minimal capacity diagnostic)
 - Compared to baselines (last-token, bigram, MLP, retrieval)
- Results
 - Linear probe accuracy: 0.18 (ours) < baselines
 - Probe collapsed to predicting [EOS]
 - Shows embeddings too general – did not encode fine-grained action semantics

Method	Val Acc	Test Acc
Last-token (baseline)	0.246	0.269
Bigram (baseline)	0.328	0.366
MLP (2×128) on embeddings	0.343	0.340
Linear probe on embeddings	0.188	0.185



Conclusion and Future works

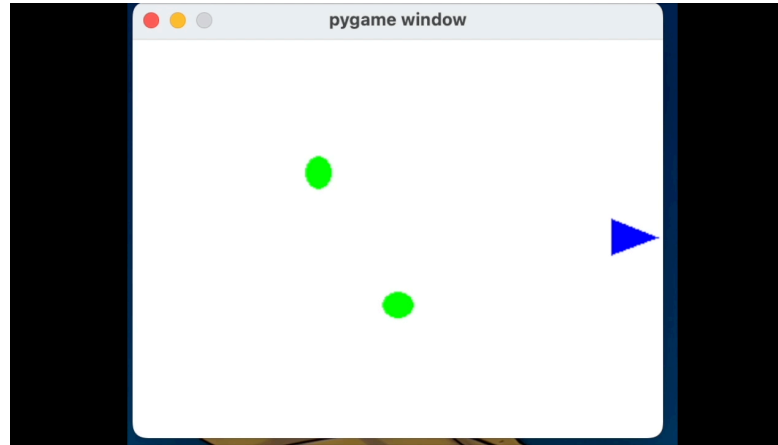
Example Subtitle

- Dual-channel design = **conceptually sound**, but not yet validated by results
- Hyperbolic embeddings failed to capture action semantics robustly
- Instant Policy baseline still stronger at **de-noising & local dynamics**
- Key bottleneck: demo quality + action clustering
- Future direction: tree-structured signatures + multi-resolution graph generation

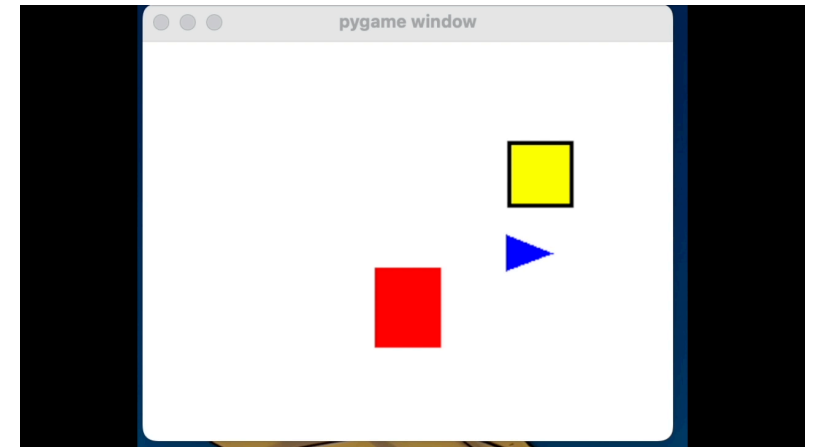
Demonstrations

My Model

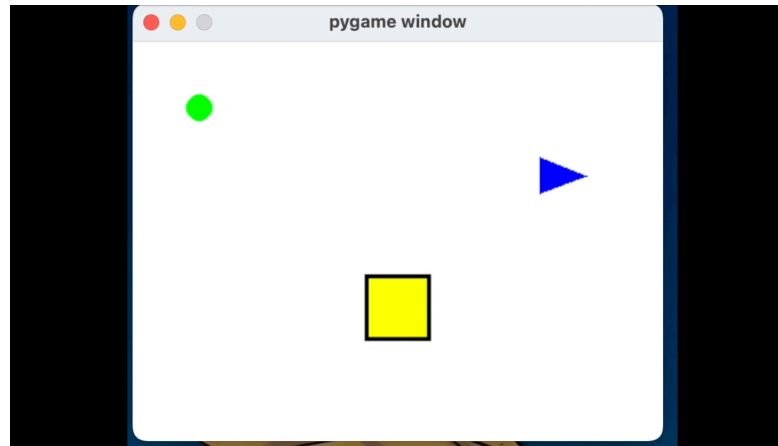
- Consist of 3 Runs:
 - Eat All : 1/3
 - Reach Goal : 1/3
 - Push & Place : 0/3
 - Park : 0/3



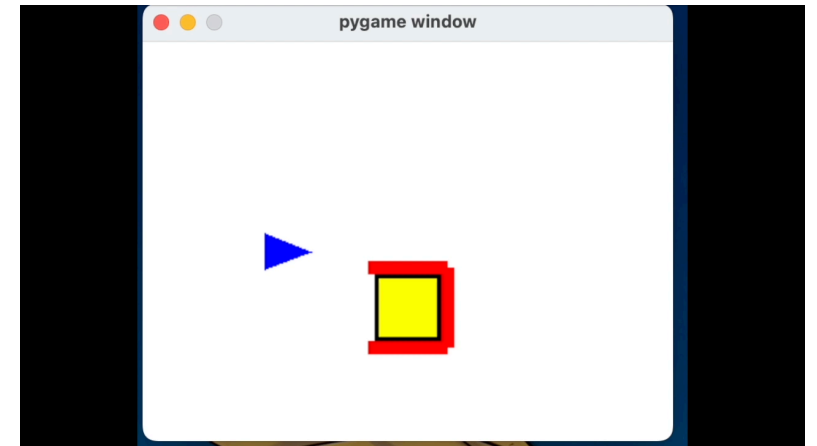
Eat All



Reach Goal



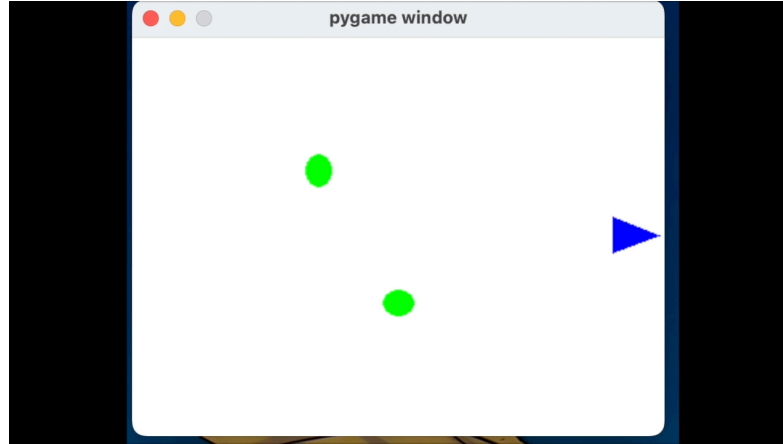
Push & Place



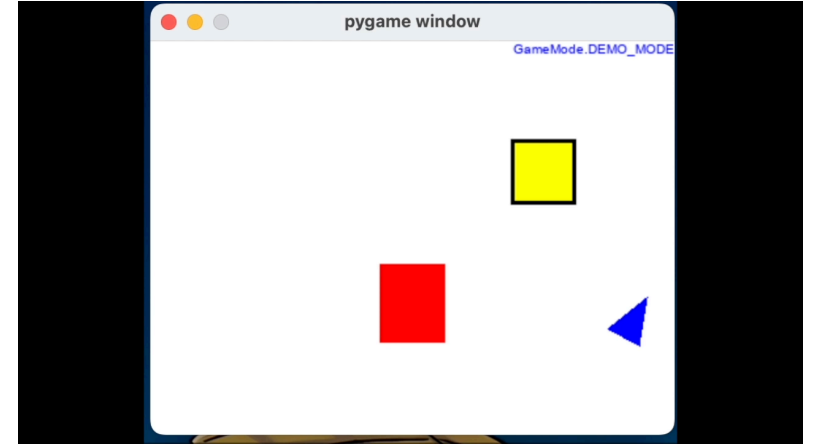
Park

Instant Policy

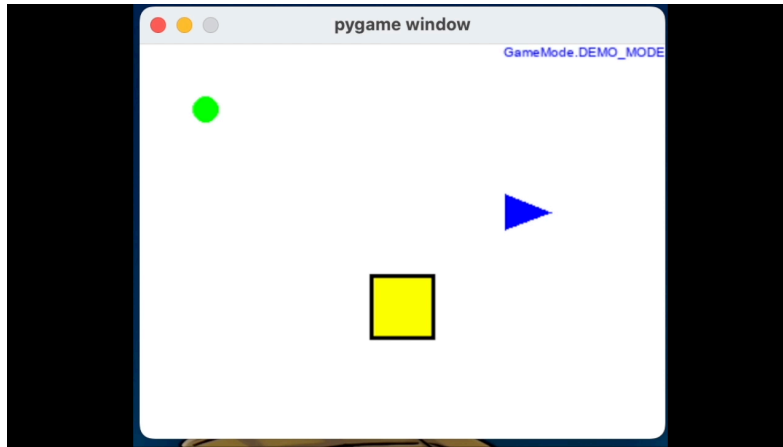
- Consist of 3 Runs:
 - Eat All : 0/3
 - Reach Goal : 1/3
 - Push & Place : 0/3
 - Park : 0/3



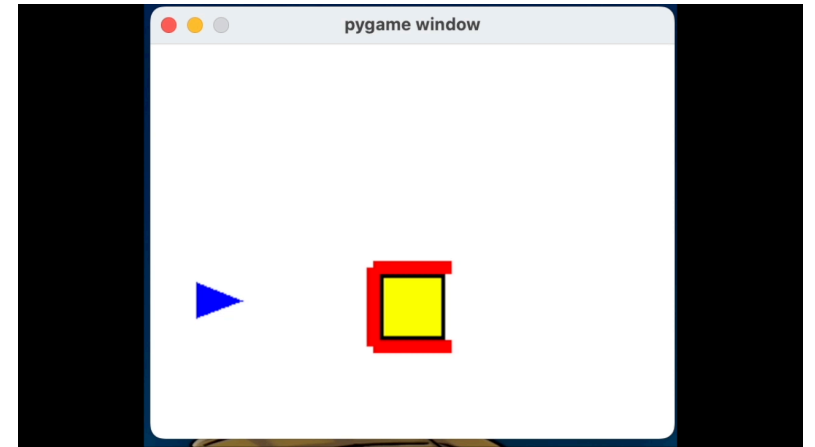
Eat All



Reach Goal



Push & Place



Park

IMPERIAL

Thank you

Acknowledgements:

Dr Edward Johns

Dr Antoine Cully

Instant Policy++

16/09/2025